Version: Beta | Updated: 2026-05-01

AIStudio

AIStudio is a private, local AI search and RAG (Retrieval-Augmented Generation) application that runs entirely on your Mac. It lets you upload your own documents, index them, and ask questions in plain English — getting cited answers grounded in your content, with no data leaving your machine and no external API or cloud dependency.

AIStudio is what you use when you want to query your own documents with AI. It is not a general-purpose chatbot — it is a document intelligence system. You bring the documents; AIStudio finds the answers.

> *AIStudio is a hands-on AI engineering lab for exploring how LLM-enabled systems behave under real operational constraints — retrieval quality, vocabulary mismatch, metadata filtering, observability, and deployment trade-offs — before those issues get hidden behind abstractions.*
>
> ***Current stack:** Python · FastAPI · local Ollama inference · Qdrant vector storage · llama3.1 · mistral · nomic-embed-text · sentence-transformers · pdfplumber page-aware PDF extraction · CrossEncoder reranker · benchmark harness · CI/CD pipeline · Docker (v3.0) · AWS ECS (v3.0)*
>
> *The architecture choices are deliberate and documented. See docs/architecture_decisions.md. For a guide to how the codebase is organized, see docs/CODEBASE_GUIDE.md.*

Point of View

Agentic AI in Financial Services: Some Reflections

This work is one aspect of the author's work in AI. It explores the transition from descriptive to generative to agentic AI, the practical constraints on autonomous systems today, and a

framework for thinking about where AI adds durable value versus where human judgment remains irreplaceable. Written December 2025.

What This Does

AIStudio gives you a **private, local search engine over your own documents** — running entirely on your Mac, with no data leaving your machine and no external API dependency.

What you actually see and do

A browser-based interface lets you manage document collections and query them conversationally. Three main areas:

- **Corpus Manager** — create named collections of documents, upload PDFs, Word docs, PowerPoints, Excel files, or Markdown, and trigger indexing. Each corpus is independently searchable.
- **Chat Interface** — ask questions in plain English and get answers grounded in your documents, with inline source citations (`[1]` , `[2]`) and a References section showing exactly which document and passage each answer came from.
- **Filters** — optional firm and year filters narrow retrieval to a specific source before the query runs. Filtering happens at the system (vector) layer — not post-hoc on results.

Two corpora ship with AIStudio, each proving something different:

Demo corpus — 20 years of original thought leadership: AIStudio ships with a curated set of 9 documents spanning 2003–2026 — IT strategy frameworks, enterprise architecture methodology, financial services technology journals, cloud migration analysis, and AI reference architecture. These are original works: articles edited for practitioner journals, engagement frameworks, and strategy documents produced across senior technology roles at major financial institutions. Querying this corpus is querying the intellectual capital of a 20-year career. The corpus and the tool are the same proof point.

A reviewer who asks *"What is the relationship between business strategy and technology strategy?"* gets a grounded, cited answer from a 2006 FS Journal article — original work, not sample data. That is what makes the demo corpus distinctive.

AIStudio as its own corpus: AIStudio's documentation — architecture decisions, benchmark methodology, retrieval guides — is available as a corpus in the standard interface. Asking *"What embedding model does AIStudio use?"* or *"How does the reranker work?"* returns cited answers from the actual codebase docs. The tool documents itself.

On performance: Warm `llama3.1:70b` and warm `llama3.1:8b` are statistically identical in query latency on Apple Silicon (~6–7s average). Once loaded into unified memory, model size stops being a latency variable. See benchmarks/ for the full benchmark harness and timestamped reports.

The QUICKSTART also shows you how to set up the **SEC 10-K corpus** to demonstrate how AIStudio can operate at scale — exploring 144 annual filings from 25 financial services firms (Goldman Sachs, JPMorgan Chase, Morgan Stanley, BlackRock, and others), 105,964 chunks, ingested in 34 minutes at 54 chunks/sec on an M4 MacBook Pro. Due to their size, the files for this corpus are not shipped with the app but need to be downloaded from the SEC first. More importantly, ingesting and indexing this type of corpus provides a good opportunity to learn how to work with a large corpus.

Quickstart

See QUICKSTART.md to get a running instance in under 30 minutes.

TL;DR for experienced users:

```
# 1. Install Qdrant (not in Homebrew — binary install required)
curl -L https://github.com/qdrant/qdrant/releases/latest/download/qdrant-aarch64-apple-darw
mkdir -p ~/bin && mv qdrant ~/bin/qdrant && echo 'export PATH="$HOME/bin:$PATH"' >> ~/.zshr

# 2. Clone and set up
mkdir -p ~/Developer && cd ~/Developer
git clone https://github.com/mbarberony/AIStudio.git && cd AIStudio
./ais_install

# 3. Start everything (stops any running services first, then restarts clean)
ais_start

# 4. Open UI
open front_end/rag_studio.html
```

Current Status

Core RAG loop working end-to-end on a 106K-chunk production corpus. Qdrant vector store (Rust-based) replaced ChromaDB after ChromaDB crashed at 32K chunks. Metadata filtering (firm, year) implemented end-to-end — backend, API, and UI. CI/CD pipeline green on every push.

Working today

- Document ingestion: PDF, Word, PowerPoint, Excel, Markdown, HTML
- Vector search via Qdrant 1.17.0 with cosine similarity
- Metadata filtering — firm and year filters at the vector layer
- RAG query with inline citations and source references
- Browser UI — corpus management (create, rename, delete, stats, inspect), file upload with progress, chat with citations
- Corpus rename — renames directory, cascades corpus_metadata.yaml, triggers background re-index
- Stats panel — per-file KB sizes, last ingestion time and duration from corpus_metadata.yaml
- Auto-linkify — URLs and corpus filenames in responses rendered as clickable links
- FastAPI backend — `/ask`, `/health`, `/corpus/*` (create, rename, delete, info, upload, ingest), `/source`, `/prewarm`
- Auto-launch script — `ais_start` starts all three services (Qdrant, Ollama, FastAPI backend)
- Benchmark harness — `benchmarks/bench.py` with CLI flags, auto-generates findings
- CI/CD — GitHub Actions: lint + unit + integration tests on every push
- Developer tooling — Makefile (`make check`, `make coverage`), pre-commit hooks

Recently completed (Beta)

- CrossEncoder reranker (ms-marco-MiniLM) — fixes vocabulary mismatch ✅
- Page-aware PDF chunking via pdfplumber — page numbers in citations ✅
- PDF viewer — direct access to source page from inline citation ✅
- `--force` ingest flag — atomic wipe + clean re-index ✅
- YAML benchmark question files with corpus auto-detection ✅
- Demo corpus benchmark: 14/14 questions pass, 6.9s avg latency (M4 Max) / 26s avg (M4 Air) ✅
- Corpus rename — UI + API, background re-index, re-ingest time estimate ✅
- Ingestion metadata — last_ingested_at and duration persisted to corpus_metadata.yaml ✅
- Manifest-driven `ais_install` — adds any new command alias in one step ✅
- Help corpus search guidance — per-document routing rules injected into system prompt ✅

In progress

- Relevance threshold — discard low-scoring chunks

• XBRL noise stripping in HTML ingestion

Roadmap in a Nutshell

Beta (now)	v2.0	v3.0
✅ CrossEncoder reranker	PDF viewer click → source page	Docker + AWS ECS Fargate
✅ Page-aware PDF chunking	PDF image identification + citation	Multi-user + shared corpora
✅ PDF viewer Open ↗ links	Clean install validation	GPU inference (Inferentia2)
✅ `--force` atomic ingest	MacBook Air validation ✅	Compiled installer (.dmg)
Relevance threshold	Benchmark comparison tooling	urCrew integration

See docs/PRODUCT_ROADMAP.md for the full phased plan. Releases go Beta → v2.0 → v3.0. There is no v1.0 by design.

Architecture — Local (Current)

[Architecture diagram]

Why Qdrant over ChromaDB:

	ChromaDB	Qdrant
Stability at scale	Crashed at 32K chunks	Stable at 106K chunks
Language	Python	Rust
Metadata filtering	Limited	Native Filter/FieldCondition
Memory model	Python GC	Rust ownership, near-zero overhead
Production path	Single-node	Sharding, replication, quantization
gRPC	No	Yes (port 6334)

Architecture — Cloud (v3.0 Roadmap)

The local four-process pattern maps directly to a containerized cloud deployment. Each process becomes a container; Qdrant and Ollama have official Docker images.

[Architecture diagram]

Local → cloud (Release 2.0) is a container boundary, not an architecture change. Same FastAPI app, same Qdrant queries, same Ollama interface — wrapped in Docker and deployed to ECS Fargate. No code changes required.

Performance Findings

Synthesized from 15 benchmark runs on MacBook Pro M4 Max (128GB unified memory):

- **Sub-7s latency** per query once the model is warm — including complex multi-source synthesis
- **99.1% pass rate** across 126 Q/A pairs (9 runs × 14 questions, identical conditions)
- **Model size does not predict warm latency** — llama3.1:70b and llama3.1:8b both land at 6.9–7.2s; the bottleneck is output token generation, not parameter count
- **Retrieval adds ~0.3–0.5s** even at 105,964 chunks — inference, not retrieval, is the bottleneck
- **Stable across successive runs** — no thermal throttling or memory pressure observed

Testing spans the Apple Silicon performance spectrum: the M4 Max (128GB) establishes the baseline above. The same system runs correctly on an M4 Air — the other end of the Apple Silicon range — with approximately 40% higher latency under equivalent conditions. Systematic benchmark data on the Air is being collected. The goal is to characterize behavior across the full range of likely deployment hardware, not just optimal conditions.

→ Benchmark reports and question sets

Benchmark

```
Corpus:     144 SEC 10-K filings, 25 financial services firms
Chunks:     105,964
Ingest:     34 min, 54 chunks/sec, 0 failures
Latency:    ~6-7s warm (8b and 70b identical on Apple Silicon)
```

```
Filtering:  firm + year metadata filters, zero latency overhead
ChromaDB:   crashed at 32,285 chunks
Qdrant:     stable at 105,964 chunks
```

See benchmarks/ for the full benchmark harness, timestamped reports, and question sets.

Manuel Barbero · mbarberony@gmail.com · linkedin.com/in/mbarberony · github.com/mbarberony/AIStudio